Chapter 3. Sharing data between threads

This chapter covers

- Problems with sharing data between threads
- Protecting data with mutexes
- Alternative facilities for protecting shared data

One of the key benefits of using threads for concurrency is the potential to easily and directly share data between them, so now that we've covered starting and managing threads, let's look at the issues surrounding shared data.

Imagine for a moment that you're sharing an apartment with a friend. There's only one kitchen and only one bathroom. Unless you're particularly friendly, you can't both use the bathroom at the same time, and if your roommate occupies the bathroom for a long time, it can be frustrating if you need to use it. Likewise, though it might be possible to both cook meals at the same time, if you have a combined oven and grill, it's just not going to end well if one of you tries to grill some sausages at the same time as the other is baking a cake. Furthermore, we all know the frustration of sharing a space and getting halfway through a task only to find that someone has borrowed something we need or changed something from the way we left it.

It's the same with threads. If you're sharing data between threads, you need to have rules for which thread can access which bit of data when, and how any updates are communicated to the other threads that care about that data. The ease with which data can be shared between multiple threads in a single process is not just a benefit—it can also be a big drawback. Incorrect use of shared data is one of the biggest causes of concurrency-related bugs, and the consequences can be far worse than sausage-flavored cakes.